Week 7 — Code With Me Lab

Stack Detectives: Exploring Memory From the Inside

Compile all programs today with:

```
gcc -o program program.c -fno-stack-protector -z execstack -no-pie -O0
```

> *These flags disable the modern defences we discussed (canary, NX, ASLR). We do this deliberately so we can observe raw stack behaviour. Never use these flags for real code.*

Part 1 — Exploring the Stack (Reading Memory)

What we're building

A function that starts at its own local variable and reads outward through memory, revealing adjacent variables, the saved BP, and the return address.

Step 1 — The base program

Type this out. Don't copy-paste — reading it as you type builds familiarity.

```
#include <stdio.h>
#include <string.h>

void explore(int size) {
    char marker = 0xAB;    /* a value we can spot easily in hex */
    char *ptr = &marker;
    int i;

    printf("\n--- Exploring %d bytes upward from &marker ---\n", size);
    for (i = 0; i < size; i++) {
        printf("%02hhx ", ptr[i]);
```

```
        if ((i + 1) % 8 == 0)
            printf("\n");
    }
    printf("\n");
}

int main(void) {
    int count = 0;
    char greeting[] = "Hello MTU";

    printf("How many bytes to explore? ");
    scanf("%d", &count);

    explore(count);
    return 0;
}
```

Compile and run. Enter `1`.

- You should see `ab` — that's your marker byte.

Run again. Enter `4`. You'll see four bytes including `ab` and whatever is in adjacent memory.

Step 2 — Print variable addresses

Add these lines to `main`, **before** calling `explore`:

```
printf("&count    = %p\n", (void*)&count);
printf("&greeting = %p\n", (void*)greeting);
```

Add these lines to `explore`, **right after** declaring `marker`:

```
printf("&marker = %p\n", (void*)&marker);
printf("&ptr    = %p\n", (void*)&ptr);
```

```
printf("&i      = %p\n", (void*)&i);
```

Compile and run with size `60`.

📋 **Write down:**

- The address of `marker`
- The address of `greeting`
- The difference between them (in decimal)

That difference is the offset from `marker` to `greeting`. Confirm: does the hex dump at that offset show `48 65 6c 6c 6f` (the ASCII bytes of `"Hello"`)?

Step 3 — Map the full layout

Using the addresses you printed, draw the stack on paper before looking at the answer below:

```
High addresses
┌──────────────────────────┐
│  main: greeting (10B)    │
│  main: count    (4B)     │
│  ??? (padding?)          │
│  return address (8B)     │  ← address of instruction after CALL explore
│  saved BP of main (8B)   │
│  explore: i     (4B)     │
│  explore: ptr   (8B)     │
│  explore: marker (1B)    │  ← ptr starts here
└──────────────────────────┘
Low addresses
```

> ***Discuss with the class:*** *The exact layout may vary — padding is inserted by the compiler for alignment. The addresses you printed tell you the real layout on your machine today.*

Step 4 — Spot the return address

Increase size until you've gone past the saved BP. You should see 8 bytes that look like a valid memory address (e.g. `...7f ff ...`). That's the return address — where `main` will jump back to after `explore` returns.

Part 2 — The Dangling Pointer Bug

What we're building

A program that returns a pointer to a local variable, then demonstrates what happens when that memory gets reused.

Step 1 — Write the program

Create a new file `dangling.c`:

```
#include <stdio.h>

int *get_secret(void) {
    int secret = 42;
    printf("[get_secret] &secret = %p, value = %d\n",
           (void*)&secret, secret);
    return &secret;   /* returning address of a local variable */
}

void overwriter(void) {
    int a = 111;
    int b = 222;
    int c = 333;
    printf("[overwriter] &a=%p  &b=%p  &c=%p\n",
           (void*)&a, (void*)&b, (void*)&c);
}
```

```c
int main(void) {
    int *p = get_secret();

    printf("\n[main] p         = %p\n", (void*)p);
    printf("[main] *p before = %d  (probably still 42)\n", *p);

    overwriter();

    printf("[main] *p after  = %d  (what happened?)\n\n", *p);

    return 0;
}
```

Step 2 — Predict before you run

Before compiling, write down your answers:

1. What will `*p` print *before* calling `overwriter` ?
2. What will `*p` print *after* calling `overwriter` ?
3. Why does the second value differ?

Step 3 — Compile and run

```
gcc -o dangling dangling.c -fno-stack-protector -O0
./dangling
```

Were your predictions correct?

📋 **Compare the addresses:**

- Address of `secret` (printed inside `get_secret`)
- Addresses of `a` , `b` , `c` (printed inside `overwriter`)

Are they the same region? Why?

Step 4 — The GCC warning

Now compile *with* warnings enabled:

```
gcc -o dangling dangling.c -fno-stack-protector -O0 -Wall
```

GCC will warn: `warning: function returns address of local variable` .

📋 **Discussion:** The program compiled anyway. The warning is there — but it's easy to miss in a large project. This is why code review and static analysis tools matter.

Step 5 — Extend it (optional challenge)

Modify `overwriter` so that its local variable `a` is initialised to the value `99` . Predict: will `*p` after the call show `99` ? Run it and find out. Then ask yourself — does the exact value depend on stack layout? Try different values.

Part 3 — Recursive Functions and the Stack

What we're building

An instrumented factorial function that prints its own stack frame address at each call depth, making the stack-building visible.

Step 1 — Write the instrumented factorial

Create `recursive.c` :

```
#include <stdio.h>

int depth = 0;   /* global, just for printing indentation */

int factorial(int n) {
```

```
    int result = 0;
    int current_depth = ++depth;

    /* print indentation to show call depth */
    int d;
    for (d = 0; d < current_depth; d++) printf("  ");
    printf("factorial(%d) entered | &n=%p | &result=%p\n",
           n, (void*)&n, (void*)&result);

    if (n <= 1) {
        for (d = 0; d < current_depth; d++) printf("  ");
        printf("factorial(%d) BASE — returning 1\n", n);
        depth--;
        return 1;
    }

    result = n * factorial(n - 1);

    for (d = 0; d < current_depth; d++) printf("  ");
    printf("factorial(%d) returning %d | &result still = %p\n",
           n, result, (void*)&result);

    depth--;
    return result;
}

int main(void) {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);

    printf("\n--- Call stack building up ---\n");
    int answer = factorial(n);
    printf("\n--- Back in main ---\n");
    printf("%d! = %d\n", n, answer);
```

```
    return 0;
}
```

Step 2 — Run with small values

Compile and run. Enter `4`.

Observe:

- Each call is indented one level deeper
- Each new `&result` address is *lower* than the previous — the stack grows downward
- On the way back out, the same `&result` address is printed — the frame is still intact until RET

📋 **Calculate:** What is the gap in bytes between `factorial(4)`'s `&result` and `factorial(3)`'s `&result`? This is the size of one stack frame.

Step 3 — Find the stack overflow limit

Increase n until the program crashes with a **segmentation fault**.

```
Enter n: 10000
```

📋 **Discussion:** The stack has a fixed maximum size (typically 8MB on Linux). Each recursive call adds a frame. Deep enough recursion exhausts the stack — this is a **stack overflow**. It's not just an error name — it's a literal overflow of the stack memory region.

Step 4 — Why iteration avoids this

Discuss: how would you rewrite `factorial` as a loop? A loop reuses the *same* stack frame — no new frame is added per iteration. This is why deep recursion can be dangerous for large inputs, and why tail-call optimisation exists in other languages.

Part 4 — Out-of-Bounds Write and Buffer Overflow

What we're building

A controlled demonstration of a buffer overflow — we overflow a small buffer and observe (and then control) what gets overwritten.

Step 1 — Observe an out-of-bounds write

Create `overflow.c` :

```
#include <stdio.h>
#include <string.h>

void vulnerable(void) {
    char buffer[16];
    int sentinel = 0xDEADBEEF;

    printf("[before] sentinel = 0x%x\n", sentinel);
    printf("[before] &buffer  = %p\n", (void*)buffer);
    printf("[before] &sentinel= %p\n", (void*)&sentinel);

    printf("Enter input: ");
    scanf("%s", buffer);    /* no length check */

    printf("[after]  sentinel = 0x%x\n", sentinel);
    printf("[after]  buffer   = \"%s\"\n", buffer);
}

int main(void) {
    vulnerable();
    printf("[main] returned normally\n");
```

```
    return 0;
}
```

Step 2 — Predict the layout

Before running, look at the stack layout of `vulnerable`:

- `buffer` is 16 bytes
- `sentinel` is 4 bytes
- Which one is at a lower address? (Hint: compile and print both addresses to find out)

📋 **Predict:** If you enter exactly 16 characters, what happens to `sentinel`? What about 20 characters?

Step 3 — Run with safe input

```
Enter input: HelloWorld
```

`sentinel` should still be `0xDEADBEEF`. All good.

Step 4 — Overflow into sentinel

Run again. Enter exactly 20 characters (16 to fill buffer + 4 to overwrite sentinel):

```
Enter input: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

Actually start with 17, then 20, then 24. Watch what happens to `sentinel`.

📋 **Discussion:**

- At what input length does `sentinel` change?
- What value does it become? (`0x41` is the ASCII code for `'A'`)
- What does this tell us about the stack layout?

Step 5 — Overflow past the sentinel

Enter a very long string (30+ characters). You will likely see a **segmentation fault** — you've overwritten the return address with `'A'` bytes (`0x4141414141414141`), and when `vulnerable` tries to return, the CPU tries to jump to address `0x4141414141414141` which is not a valid instruction address.

📋 **Discuss:**

- What exactly caused the segfault?
- If an attacker could control *which* address is written there, what could they do?
- What are the three defences that make this harder in practice?

Step 6 — Fix the vulnerability

Modify `vulnerable` to use a length-limited input method:

```
fgets(buffer, sizeof(buffer), stdin);
```

or:

```
scanf("%15s", buffer);   /* at most 15 chars + null terminator */
```

Recompile. Verify that no matter how long the input is, `sentinel` is never overwritten.

📋 **Takeaway:** The fix is one character change or one function swap. The vulnerability is a missing constraint on input size. This is one of the most common classes of real-world security bug.

Lab Summary

Part	What you observed
Part 1	Stack frames are contiguous in memory — you can read adjacent frames
Part 2	Returning a local variable's address is silently broken — the memory is reused

Part	What you observed
Part 3	Recursion builds stack frames; deep recursion exhausts the stack
Part 4	Unbounded input overwrites adjacent variables, then the return address

The single lesson to take away: C gives you complete access to memory and complete responsibility for staying in bounds. The CPU and OS will not stop you from writing past the end of your array — and neither will C.